

Pre-Reads: GitHub Workflow & Collaboration



Nishut Suman

2 Jan, 2026 At 10:00 AM

Pre-Reads

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Pre-Read: GitHub Workflow & Collaboration

What You'll Gain from This Pre-Read

After reading, you'll be able to:

- Connect a local Git repository to a remote repository on GitHub.
- Push and pull commits between local and remote branches.
- Create, review, and merge Pull Requests (PRs) on GitHub.
- Recognize and resolve a simple merge conflict.
- Apply a basic collaboration workflow used in real teams.

Think of this as: Learning how to put your code where teammates can see it, review it, and improve it together.

What This Pre-Read Covers

This pre-read will:

- Introduce core collaboration concepts and vocabulary
- Explain why GitHub matters for teamwork and code safety
- Show practical command examples and a minimal PR workflow
- Prepare you for the hands-on "Push to GitHub" exercise

Part 1: The Big Picture - Why Does This Matter?

Opening Hook:

Imagine this - you've just finished a new feature on your laptop. Now you want your teammate to review it,

suggest improvements, and merge it safely into the main project - without emailing files or risking overwriting each other's code.

- **That's where GitHub comes in.**

Expanded View:

GitHub acts as the collaborative hub built on top of **Git**.

It's where your local work goes online, allowing your whole team to:

- Store and back up code securely in the cloud.
- Review each other's changes through Pull Requests (PRs)
- Discuss improvements and track version history
- Merge updates safely into the shared main branch
- **In short**, GitHub turns solo coding into team collaboration - helping developers work together efficiently and confidently.

Where You'll Use This:

Job roles:

- **Software Engineer** - daily code collaboration and release workflows.
- **QA/Tester or DevOps** - review changes and verify readiness before deployment.

Real products:

- GitHub (host & PR system)
- Git (local version control tool)
- CI tools (e.g., GitHub Actions) that run tests on PRs

What you can build:

- Team projects with many contributors
- Open-source contributions and portfolios
- Safe deployment pipelines with code review gates

Think of it like this:

Git is your personal notebook; **GitHub** is the shared team whiteboard where everyone posts proposals and comments.

Limitation:

The whiteboard analogy breaks for access control and CI automation - GitHub also manages permissions and automated checks which a simple whiteboard can't.

Part 2: Your Roadmap Through This Topic

Here's what we'll explore together:

1. Remotes

You'll learn what a remote repository is and how to connect a local repo to GitHub.

2. Push & Pull

You'll discover how to send commits up to GitHub and fetch others' commits down to your machine.

3. Collaboration Workflow (Pull Requests)

You'll learn the typical PR lifecycle:

branch → push → open PR → review → update → merge.

4. Merge Conflicts

You'll see a simple example of how conflicts happen and the basic steps to resolve them.

5. Practical Exercise

A short, hands-on plan:

Create a GitHub repo, add a remote, and push your main branch.

The journey:

Start connecting your repo to GitHub, practice exchanging changes, and finish by confidently opening and merging a pull request.

Part 3: Key Terms to Listen For

Remote

A hosted copy of your repository (e.g., on **GitHub**) that you can push to and pull from.

Example:

origin is a common name for the **main** remote.

Push

Sending your local commits to a remote repository.

Think of it as: Uploading your changes to the team server.

Pull

Fetching changes from a remote and integrating them into your current branch.

In practice:

git pull origin main gets and merges remote changes.

Branch

An independent line of development. Use branches per feature or fix.

Example:

feature/login keeps work isolated from **main**.

Pull Request (PR)

A request on GitHub to merge one branch into another, usually **main**. It enables review and discussion.

Merge Conflict

When Git can't automatically combine changes because the same lines were edited differently.

Example: Two people edited the same line in README.md. (a conflict can happen in any file)

💡 **Key Insight:** Branches + PRs provide a safe, reviewable path to get code from a developer's machine into the project's main branch.

Part 4: Concepts in Action

Seeing Remotes, Push, Pull, and a PR in Action

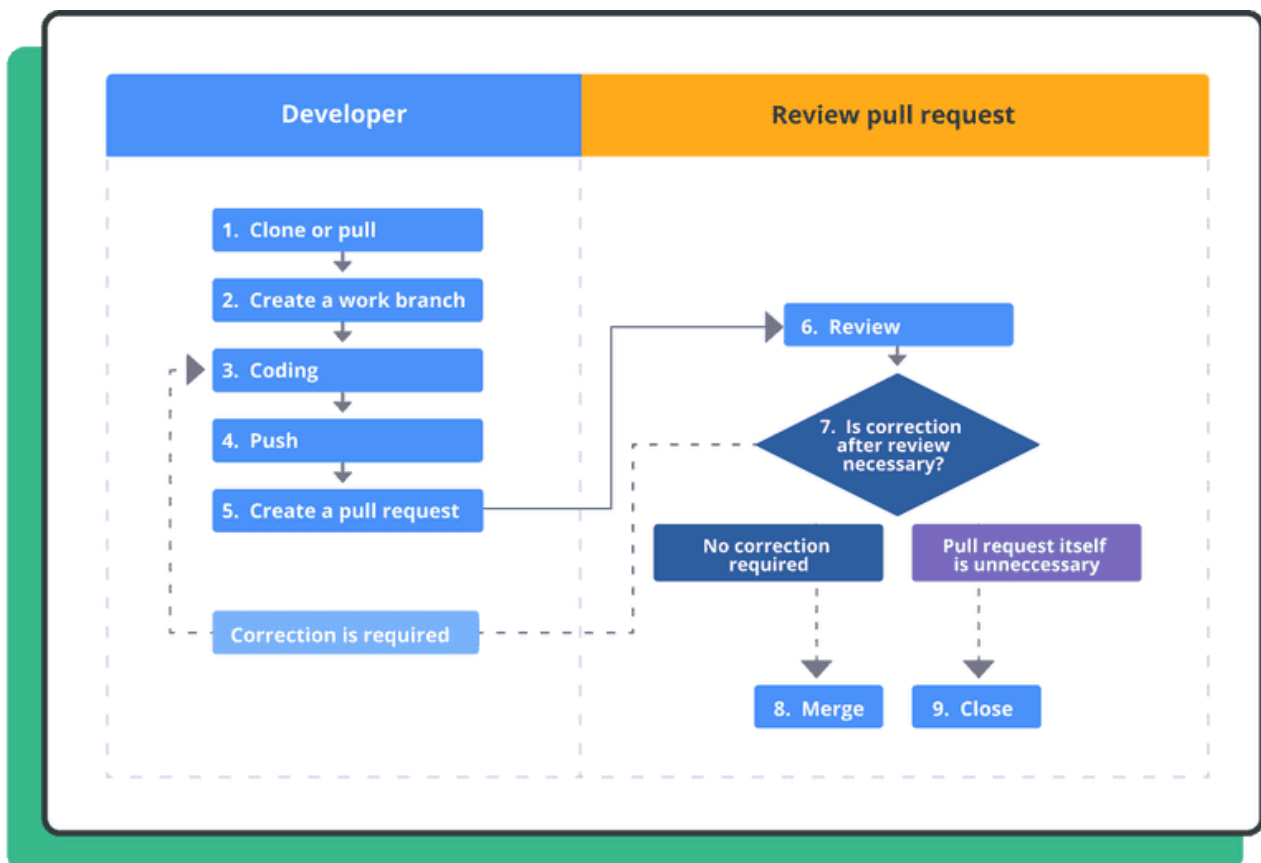


Figure 1: Typical GitHub collaboration workflow - developer pushes a branch, opens a Pull Request, and merges after review.

The scenario: You finished changes locally on branch **feature-x**.
Now you want teammates to review and merge them into **main**.

Our approach: Add GitHub as a remote, push the feature branch, open a PR on GitHub, address review comments, and merge when approved.

The code:

```
# Step 1: Connect to GitHub (run once per repo)
git remote add origin https://github.com/your-username/your-repo.git
# Verify remote
git remote -v

# Step 2: Push your main branch (first push may require setting upstream)
git push -u origin main

# Step 3: Create and push a feature branch
git checkout -b feature-x
# ...make changes, then:
git add .
git commit -m "Add feature X"
git push -u origin feature-x

# Step 4: On GitHub: open a Pull Request from feature-x -> main
# Step 5: After review, sync changes if reviewers requested edits:
# Make edits locally, commit, then:
git push origin feature-x

# Step 6: Merge the PR on GitHub (or from CLI)
# Optional: delete the branch after merge
git push origin --delete feature-x
```

Quick Example — Resolving a Simple Merge Conflict

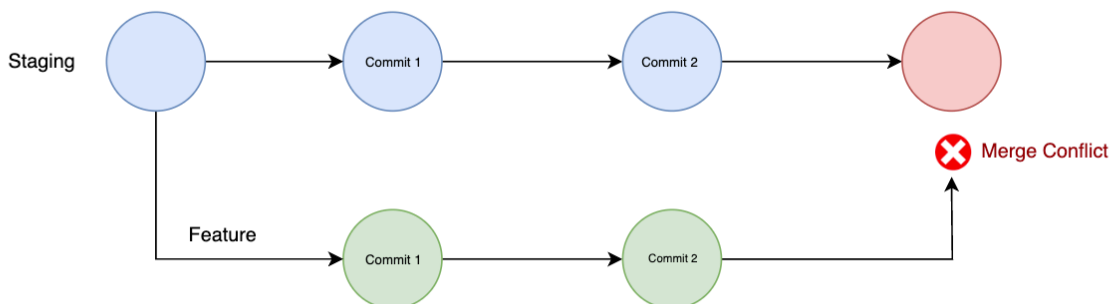


Figure 2: A visual example showing how Git highlights conflicting sections when two edits overlap.

Key takeaway: Conflicts require human judgment - choose which changes to keep, possibly combine both, then commit the resolved file.

Part 5: How This Topic Connects

Builds on:

- Local Git basics (commits, branches)
- Command-line familiarity (or a Git GUI)

Enables:

- Safe team collaboration with review history
- Participation in open-source projects and team workflows

Related concepts you might explore:

- Continuous Integration (CI) - Runs automated tests on PRs
 - Protected Branches - Enforcing reviews before merges
 - Forking workflows - Common in open-source contributions
-

Part 6: Questions to Keep in Mind

1. How should my team decide branch naming and PR rules?
2. When is it OK to merge without review?
3. What does a failing CI check mean for a PR and who fixes it?

Reflect: Which parts of your current workflow introduce the most friction - communication, testing, or merge conflicts?

How to Read These Notes Effectively

DO:

Read actively, try the **git** commands in a sandbox repo, and open a PR on a test repository.

DON'T:

Try to memorize everything - focus on the **flow** and why each step exists.

What's Next?

Explore deeper: Create a test repo on GitHub and practice pushing a feature branch and opening a PR.

Practice: Pair up with a peer and exchange small PRs to review.

Continue learning: Next: Protected branches, CI integration, and review etiquette.

Additional resources:

- <https://docs.github.com/en> (official guides for PRs and remotes)
- <https://git-scm.com/book/en/v2> (deep reference with examples)

Final Thought

You've built the bridge between your laptop and the team's codebase - keep practicing the branch → PR → review loop, and you'll be a reliable collaborator in any team.
